

# TASKFLOW

Sistema de Gestão de Tarefas Colaborativo

**Documento do Projeto**

Versão 1.1  
5 de julho de 2026

# Sumário

---

## 1. Introdução

## 2. Objetivos

### 2.1 Objetivo Geral

### 2.2 Objetivos Específicos

## 3. Escopo do Projeto

### 3.1 Dentro do Escopo

### 3.2 Fora do Escopo

## 4. Arquitetura da Solução

### 4.1 Camadas da Aplicação (Backend)

### 4.2 Descrição das Camadas

### 4.3 Fluxo de Dados (exemplo: criar tarefa)

### 4.4 Arquitetura do Frontend

### 4.5 Tempo Real (WebSocket)

## 5. Stack Tecnológica

## 6. Estrutura de Pastas do Repositório

## 7. Boas Práticas Adotadas

### 7.1 Princípios SOLID

### 7.2 Padrões de Projeto

### 7.3 Estratégia de Testes

### 7.4 CI/CD

## 8. Roadmap de Implementação

## 9. Riscos e Mitigações

## 10. Critérios de Sucesso

## 11. Considerações Finais

## 12. Histórico de Versões

# 1. Introdução

---

O TaskFlow é uma plataforma web fullstack para gestão colaborativa de tarefas, inspirada em ferramentas como Trello e Jira, porém com escopo simplificado e foco didático em arquitetura de software. O projeto foi concebido para servir como referência de implementação seguindo princípios de Clean Architecture, SOLID, testes automatizados em múltiplas camadas e um pipeline de CI/CD básico.

Este documento descreve a visão geral da solução, a arquitetura escolhida, a stack tecnológica, a estrutura do repositório, as boas práticas de engenharia adotadas e o roadmap de implementação. O detalhamento de requisitos funcionais e não funcionais está no Documento de Requisitos, que acompanha este material.

## 2. Objetivos

---

### 2.1 Objetivo Geral

Desenvolver uma aplicação fullstack completa que permita a equipes organizarem e acompanharem o progresso de tarefas em quadros colaborativos, servindo como exemplo de arquitetura limpa e boas práticas de engenharia de software.

### 2.2 Objetivos Específicos

1. Permitir que usuários criem contas, façam login e gerenciem projetos (quadros) de forma isolada por organização/equipe.
2. Permitir a criação de quadros com colunas customizáveis (ex.: A Fazer, Em Andamento, Concluído).
3. Permitir a criação, edição, atribuição, priorização e movimentação de tarefas (cards) entre colunas.
4. Suportar comentários e histórico de atividades em cada tarefa.
5. Aplicar arquitetura em camadas com separação clara de responsabilidades (SOLID).
6. Garantir cobertura de testes automatizados (unitários, integração e end-to-end) nas camadas críticas.
7. Automatizar build, lint, testes e deploy via pipeline de CI/CD.

## 3. Escopo do Projeto

---

### 3.1 Dentro do Escopo

- Autenticação e autorização de usuários (JWT + refresh token).
- CRUD de organizações, quadros, colunas, tarefas e comentários.
- Atribuição de responsáveis e prazos em tarefas.
- Movimentação de tarefas entre colunas (drag-and-drop no frontend).
- Filtros e busca de tarefas por responsável, status e prazo.
- Painel de atividades/histórico por tarefa.
- API REST documentada (OpenAPI/Swagger).
- Testes automatizados e pipeline de CI/CD.
- Notificações em tempo real via WebSocket **v1.1**.
- Convite pendente para e-mails sem conta, com auto-associação no cadastro **v1.1**.

A versão 1.0 deste documento listava "notificações em tempo real via WebSocket" como fora do escopo inicial. Na prática, essa capacidade foi implementada logo após a Fase 8 (ver Documento de Requisitos, changelog v1.6) e passa a constar aqui dentro do escopo — ver seção 12 para o histórico completo desta mudança.

### 3.2 Fora do Escopo

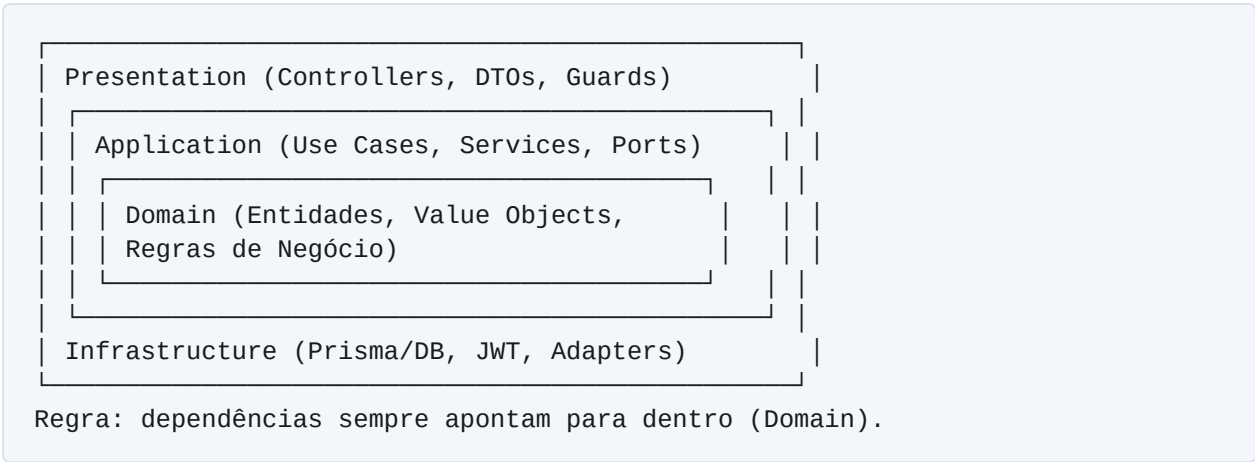
- ~~Notificações em tempo real via WebSocket~~ — implementado, ver 3.1.
- Aplicativo mobile nativo.
- Integrações com ferramentas externas (Slack, GitHub, e-mail).
- Relatórios avançados e dashboards analíticos.

## 4. Arquitetura da Solução

---

A solução adota os princípios de Clean Architecture, organizando o backend em camadas concêntricas, onde as regras de negócio (domínio) não dependem de frameworks, banco de dados ou detalhes de infraestrutura. As dependências apontam sempre para dentro, em direção ao domínio.

### 4.1 Camadas da Aplicação (Backend)



### 4.2 Descrição das Camadas

| Camada         | Responsabilidade                                                                                                    |
|----------------|---------------------------------------------------------------------------------------------------------------------|
| Domain         | Entidades de negócio, value objects e regras invariantes, sem dependência de frameworks.                            |
| Application    | Casos de uso (use cases) que orquestram o domínio; define interfaces (ports) para persistência e serviços externos. |
| Infrastructure | Implementações concretas das interfaces: repositórios Prisma, serviço de hashing, gateway de WebSocket, etc.        |
| Presentation   | Controllers REST, DTOs de entrada/saída, validação, guards de autenticação/autorização.                             |

### 4.3 Fluxo de Dados (exemplo: criar tarefa)

1. O Controller recebe a requisição HTTP e valida o DTO de entrada.
2. O Controller chama o Use Case correspondente (CreateTaskUseCase), injetado via interface.
3. O Use Case aplica as regras de negócio do Domain (ex.: validar limites de tarefas por coluna).
4. O Use Case chama o repositório (interface) para persistir a entidade.
5. A implementação concreta do repositório (Infrastructure/Prisma) executa a operação no PostgreSQL.
6. O resultado é mapeado para um DTO de resposta e devolvido ao cliente.

### 4.4 Arquitetura do Frontend

O frontend segue uma organização por features (feature-based folder structure), com separação entre camadas de apresentação (componentes React), estado (stores) e comunicação com a API (services), evitando acoplamento direto entre componentes de UI e chamadas HTTP.

## 4.5 Tempo Real (WebSocket) v1.1

Um módulo de infraestrutura dedicado ( `RealtimeModule` ) expõe um gateway WebSocket (namespace `/realtime` ), autenticado com o mesmo access token JWT usado no REST. Os Use Cases de tarefas e colunas notificam o gateway via uma porta ( `RealtimeNotifier` ) após persistir uma mudança — o mesmo padrão de ports/adapters já usado para persistência, mantendo o domínio e a camada de aplicação livres de qualquer dependência de Socket.io. O frontend entra na room de um quadro ao abri-lo e recarrega os dados ao receber qualquer evento, sem necessitar de F5.

## 5. Stack Tecnológica

| Camada                           | Tecnologia                              | Justificativa                                                                                                             |
|----------------------------------|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Frontend                         | React 18 + TypeScript + Vite            | Tipagem estática, build rápido e ecossistema maduro para testes.                                                          |
| Estado (Front)                   | Zustand                                 | Gerenciamento de estado leve, sem boilerplate excessivo do Redux.                                                         |
| Testes (Front)                   | Vitest + React Testing Library          | Testes rápidos e focados em comportamento do usuário.                                                                     |
| Backend                          | Node.js + NestJS                        | Estrutura modular nativa que favorece Clean Architecture, DI e SOLID.                                                     |
| Testes (Back)                    | Jest + Supertest                        | Padrão de mercado para testes unitários e de integração em Node.                                                          |
| Banco de Dados                   | PostgreSQL                              | Banco relacional robusto, ideal para dados estruturados e relacionamentos.                                                |
| ORM                              | Prisma                                  | Type-safety no acesso a dados e migrations versionadas.                                                                   |
| Autenticação                     | JWT (access + refresh token)            | Padrão stateless amplamente adotado para APIs REST.                                                                       |
| Tempo real <span>v1.1</span>     | Socket.io (WebSocket)                   | Integração madura com NestJS via <code>@nestjs/websockets</code> , com fallback de transporte e rooms nativas por quadro. |
| Logging <span>v1.1</span>        | nestjs-pino                             | Logs estruturados em JSON, com redação automática de dados sensíveis (senhas, tokens).                                    |
| Segurança HTTP <span>v1.1</span> | Helmet + <code>@nestjs/throttler</code> | Cabeçalhos de segurança padrão e rate limiting contra força bruta/spam de cadastro.                                       |

|                     |                         |                                                                     |
|---------------------|-------------------------|---------------------------------------------------------------------|
| Containerização     | Docker + docker-compose | Ambiente de desenvolvimento e produção reproduzível.                |
| CI/CD               | GitHub Actions          | Integração nativa com repositório, pipelines simples de configurar. |
| Documentação da API | Swagger/OpenAPI         | Documentação interativa gerada a partir do código.                  |

## 6. Estrutura de Pastas do Repositório

```

taskflow/
├── backend/
│   ├── src/
│   │   ├── modules/
│   │   │   ├── auth/
│   │   │   ├── users/
│   │   │   ├── boards/
│   │   │   ├── columns/
│   │   │   ├── realtime/      # gateway WebSocket (v1.1)
│   │   │   └── tasks/
│   │   │       ├── domain/    # entidades, VOs
│   │   │       ├── application/ # use cases, ports
│   │   │       ├── infrastructure/ # repositórios Prisma
│   │   │       └── presentation/ # controllers, DTOs
│   │   ├── shared/           # utilitários, exceptions
│   │   └── main.ts
│   ├── scripts/              # load-test.mjs (RNF-003)
│   ├── test/                  # testes e2e
│   ├── prisma/                # schema.prisma, migrations
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   ├── boards/
│   │   │   └── tasks/
│   │   ├── shared/           # componentes e hooks comuns
│   │   └── services/         # clients HTTP + WebSocket (API)
│   └── Dockerfile
├── docs/                      # documentos do projeto
├── .github/workflows/         # pipelines de CI/CD
├── docker-compose.yml
├── docker-compose.staging.yml
└── README.md

```

## 7. Boas Práticas Adotadas

### 7.1 Princípios SOLID

| Princípio                 | Aplicação no TaskFlow                                                                                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| S — Single Responsibility | Cada Use Case executa uma única ação de negócio (ex.: CreateTaskUseCase, MoveTaskUseCase).                                                        |
| O — Open/Closed           | Novas regras de notificação ou validação são adicionadas via novas implementações de interfaces, sem alterar código existente.                    |
| L — Liskov Substitution   | Qualquer implementação de um repositório (ex.: InMemoryTaskRepository em testes) pode substituir a implementação Prisma sem quebrar os Use Cases. |
| I — Interface Segregation | Interfaces (ports) pequenas e específicas, como TaskRepository e RealtimeNotifier, em vez de uma interface genérica única.                        |
| D — Dependency Inversion  | Use Cases dependem de abstrações (ports), e as implementações concretas são injetadas via container de DI do NestJS.                              |

### 7.2 Padrões de Projeto

- Repository Pattern — abstrai o acesso a dados do domínio.
- Dependency Injection — via decorators e módulos do NestJS.
- DTO Pattern — separação entre modelos de domínio e contratos de API.
- Factory — criação de entidades complexas de domínio.
- Strategy — implementação intercambiável de regras de notificação/validação futuras.

### 7.3 Estratégia de Testes

A pirâmide de testes é aplicada da seguinte forma:

| Tipo       | Ferramenta       | Escopo                                                                                | Meta de Cobertura |
|------------|------------------|---------------------------------------------------------------------------------------|-------------------|
| Unitários  | Jest / Vitest    | Domain e Application (use cases) isolados com mocks/fakes.                            | ≥ 80%             |
| Integração | Jest + Supertest | Controllers + banco de dados de teste (PostgreSQL em container).                      | Principais fluxos |
| End-to-End | Playwright       | Fluxos críticos do usuário no frontend (login, criar tarefa, mover card, tempo real). | Cenários-chave    |



|               |            |                                                                   |                          |
|---------------|------------|-------------------------------------------------------------------|--------------------------|
| Carga<br>v1.1 | autocannon | Endpoints de listagem (GET /boards, GET /tasks) sob carga normal. | p95 < 300ms<br>(RNF-003) |
|---------------|------------|-------------------------------------------------------------------|--------------------------|

## 7.4 CI/CD

O pipeline de CI/CD é executado via GitHub Actions a cada push e pull request, com os seguintes estágios:

1. Lint — verificação de estilo de código (ESLint + Prettier).
2. Testes — execução de testes unitários e de integração com relatório de cobertura.
3. Build — compilação do backend (TypeScript) e build de produção do frontend (Vite).
4. Build de imagem Docker — geração das imagens de backend e frontend.
5. Publicação — build e push das imagens para o GitHub Container Registry (branch main).
6. E2E — suíte Playwright completa contra a stack via docker-compose.

```
# .github/workflows/ci.yml (resumo)
on: [push, pull_request]
jobs:
  lint:      # eslint + prettier
  test:      # jest/vitest com cobertura
  build:     # build backend e frontend
  docker:    # build das imagens
  publish:   # push para ghcr.io (branch main)
  e2e-playwright: # suíte e2e contra docker-compose
```

## 8. Roadmap de Implementação

| Fase                       | Entregas                                                           | Duração Estimada |
|----------------------------|--------------------------------------------------------------------|------------------|
| 1. Fundação                | Setup do monorepo, Docker, CI básico, modelagem do banco (Prisma). | 1 semana         |
| 2. Autenticação            | Cadastro, login, JWT + refresh token, guards de autorização.       | 1 semana         |
| 3. Quadros e Colunas       | CRUD de organizações, quadros e colunas.                           | 1 semana         |
| 4. Tarefas                 | CRUD de tarefas, atribuição, prazos, movimentação entre colunas.   | 1,5 semana       |
| 5. Comentários e Histórico | Comentários em tarefas e log de atividades.                        | 1 semana         |

|                              |                                                                                                                                                            |            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 6. Frontend Completo         | Integração total da UI com a API, drag-and-drop.                                                                                                           | 1,5 semana |
| 7. Testes e Qualidade        | Cobertura de testes, revisão de SOLID, refino de CI/CD.                                                                                                    | 1 semana   |
| 8. Documentação e Deploy     | README final, Swagger, infraestrutura de deploy (GHCR + docker-compose.staging.yml).                                                                       | 0,5 semana |
| Pós-Fase 8 <span>v1.1</span> | RNFs pendentes (logs estruturados, responsividade, carga), hardening de segurança, paginação, convite pendente e notificações em tempo real via WebSocket. | —          |

## 9. Riscos e Mitigações

| Risco                                                 | Impacto | Mitigação                                                                                                       |
|-------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------|
| Acoplamento excessivo entre camadas                   | Alto    | Revisões de código focadas em SOLID e uso de interfaces (ports) obrigatório entre Application e Infrastructure. |
| Baixa cobertura de testes por prazo apertado          | Médio   | Definir cobertura mínima no pipeline de CI/CD (quality gate) que bloqueia merge.                                |
| Complexidade de drag-and-drop no frontend             | Médio   | Uso de biblioteca consolidada (dnd-kit) em vez de implementação própria.                                        |
| Divergência entre schema do banco e modelo de domínio | Baixo   | Uso de Prisma com migrations versionadas e mapeamento explícito domínio ↔ persistência.                         |

## 10. Critérios de Sucesso

- Todos os requisitos funcionais do Documento de Requisitos implementados e testados.
- Cobertura de testes automatizados igual ou superior a 80% nas camadas de Domain e Application.
- Pipeline de CI/CD executando lint, testes e build sem falhas na branch principal.
- README profissional permitindo que qualquer desenvolvedor suba o ambiente local em menos de 10 minutos.
- Arquitetura validada sem violações relevantes dos princípios SOLID em revisão de código.

## 11. Considerações Finais

---

Este documento serve como guia arquitetural e de planejamento para a implementação do TaskFlow. O detalhamento de cada requisito, incluindo critérios de aceitação, modelo de dados e especificação de API, está descrito no Documento de Requisitos (disponível em .pdf e .json, este último otimizado para consumo por agentes de IA durante a implementação assistida).

## 12. Histórico de Versões

---

| Versão | Data       | Resumo das mudanças                                                                                                                                                                                                                                                                                                                                                                                         |
|--------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0    | 2026-07-03 | Versão inicial do documento do projeto.                                                                                                                                                                                                                                                                                                                                                                     |
| 1.1    | 2026-07-05 | Movida "notificações em tempo real via WebSocket" de Fora do Escopo para Dentro do Escopo (implementada pós-Fase 8). Adicionado convite pendente ao escopo. Stack tecnológica atualizada com Socket.io, nestjs-pino, Helmet e @nestjs/throttler. Estrutura de pastas atualizada com o módulo realtime/. Estratégia de testes ganha a linha de teste de carga (RNF-003). Roadmap ganha a linha "Pós-Fase 8". |

---